Masterclass: Spring Boot RESTful API (Part 3: Advanced Level)

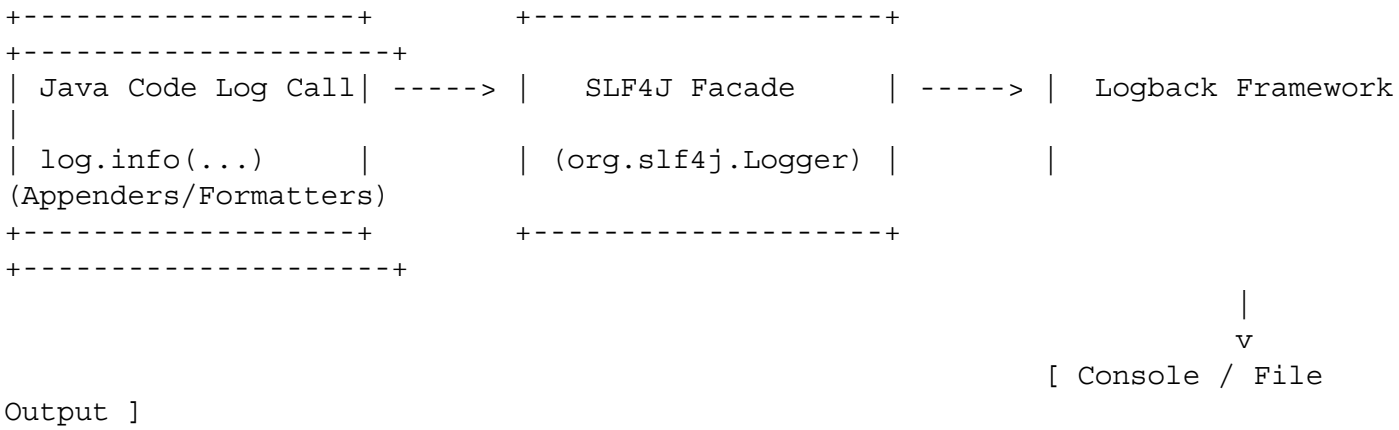
We will elevate our **Employee Management System (EMS)** into a secure, scalable, fully documented, and production-ready microservice module.

36. Logging (SLF4J + Logback)

Definition & Purpose

Logging is the practice of recording operational events, errors, and informational messages generated by an application during execution. Spring Boot uses **SLF4J (Simple Logging Facade for Java)** as an abstraction layer with **Logback** as the default underlying implementation.

Internal Working & Flow



When a log method is invoked, SLF4J checks the configured log level threshold (TRACE < DEBUG < INFO < WARN < ERROR). If the level is enabled, Logback formats the message using the configured pattern and dispatches it to configured **Appenders** (Console, File, Rolling File, or Remote Syslog).

Complete Implementation & Source Code

src/main/resources/logback-spring.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
    <property name="LOG_PATH" value="./logs"/>

    <appender name="Console" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36}
- %msg%n</pattern>
        </encoder>
```

```
    </appender>

    <appender name="RollingFile"
class="ch.qos.logback.core.rolling.RollingFileAppender">
        <file>${LOG_PATH}/ems-application.log</file>
        <encoder>
            <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36}
- %msg%n</pattern>
        </encoder>
        <rollingPolicy
class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">

<fileNamePattern>${LOG_PATH}/archived/ems-%d{yyyy-MM-dd}.%i.log</fileNamePatter
n>
            <timeBasedFileNamingAndTriggeringPolicy
class="ch.qos.logback.core.rolling.SizeAndTimeBasedFNATP">
                <maxFileSize>10MB</maxFileSize>
            </timeBasedFileNamingAndTriggeringPolicy>
            <maxHistory>30</maxHistory>
        </rollingPolicy>
    </appender>

    <root level="INFO">
        <appender-ref ref="Console"/>
        <appender-ref ref="RollingFile"/>
    </root>
</configuration>
```

EmployeeServiceLoggingImpl.java:

```
package com.edtech.ems.service;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;

@Service
public class EmployeeServiceLoggingImpl {

    private static final Logger log =
LoggerFactory.getLogger(EmployeeServiceLoggingImpl.class);

    public void processEmployeePayroll(Long employeeId) {
        log.info("Starting payroll processing for employee ID: {}",
employeeId);
        try {
            // Business logic execution
            log.debug("Calculating tax brackets and deductions for employee ID:
{}", employeeId);
        } catch (Exception ex) {
            log.error("Critical error during payroll processing for employee
```

```
ID: {}", employeeId, ex);
            throw ex;
        }
        log.info("Successfully completed payroll processing for employee ID:
{}", employeeId);
    }
}
```

Production & Troubleshooting

- **Best Practice**: Use parameterized log messages (log.info("User: {}", userId)) instead of string concatenation (log.info("User: " + userId)) to prevent unnecessary String object creation when logging is disabled.
- **Production Recommendation**: Output logs in **JSON format** (e.g., using logstash-logback-encoder) to enable seamless parsing by centralized log aggregation platforms like ELK Stack (Elasticsearch, Logstash, Kibana) or Datadog.

37. Lombok

Definition & Purpose

Project Lombok is a Java library that Plugs into your IDE and build tools to automatically generate boilerplate code such as getters, setters, constructors, equals(), hashCode(), and builder patterns at compile time via annotation processing.

Internal Working

Lombok hooks into the Java Compiler (javac) via the **Annotation Processing Tool (APT)** API (JSR 269). During compilation, Lombok modifies the Abstract Syntax Tree (AST) before standard byte-code generation (.class files). The compiled byte-code contains the generated getters/setters directly, meaning there is zero runtime reflection penalty.

Complete Source Code

```
package com.edtech.ems.dto;

import lombok.*;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class EmployeeLombokDTO {
    private Long id;
    private String firstName;
    private String lastName;
    private String email;
    private String department;
    private Double salary;
```

```
}
```

Advantages & Pitfalls

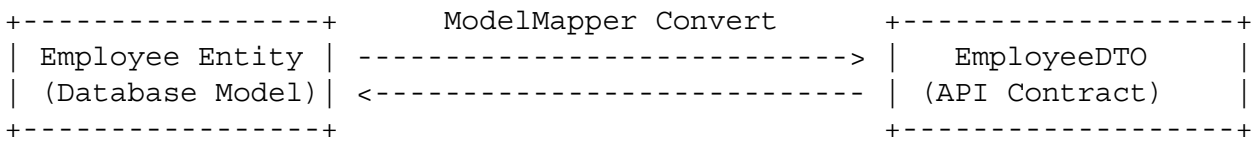
- **Advantages**: Cuts boilerplate code by up to 70%, improves readable entity/DTO definitions.
- **Disadvantage / Common Error**: Using @Data directly on JPA Entity objects can lead to infinite recursion errors in hashCode() and toString() when bidirectional @OneToMany / @ManyToOne relationships exist.
- **Best Practice**: Use @Getter, @Setter, and @ToString(exclude = "relationshipField") explicitly on JPA Entities instead of @Data.

38 & 39. DTO Pattern & ModelMapper

Definition & Purpose

The **Data Transfer Object (DTO)** pattern encapsulates data transferred between systems or application layers to hide domain implementation details and optimize network payloads. **ModelMapper** is an object-mapping framework that automates the copy of property values from domain Entities to DTOs and vice versa.

Architecture Mapping Flow



Complete Source Code Configuration & Usage

ModelMapperConfig.java:

```
package com.edtech.ems.config;

import org.modelmapper.ModelMapper;
import org.modelmapper.convention.MatchingStrategies;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class ModelMapperConfig {

    @Bean
    public ModelMapper modelMapper() {
        ModelMapper mapper = new ModelMapper();
        mapper.getConfiguration()
              .setMatchingStrategy(MatchingStrategies.STRICT)
              .setFieldMatchingEnabled(true)
```

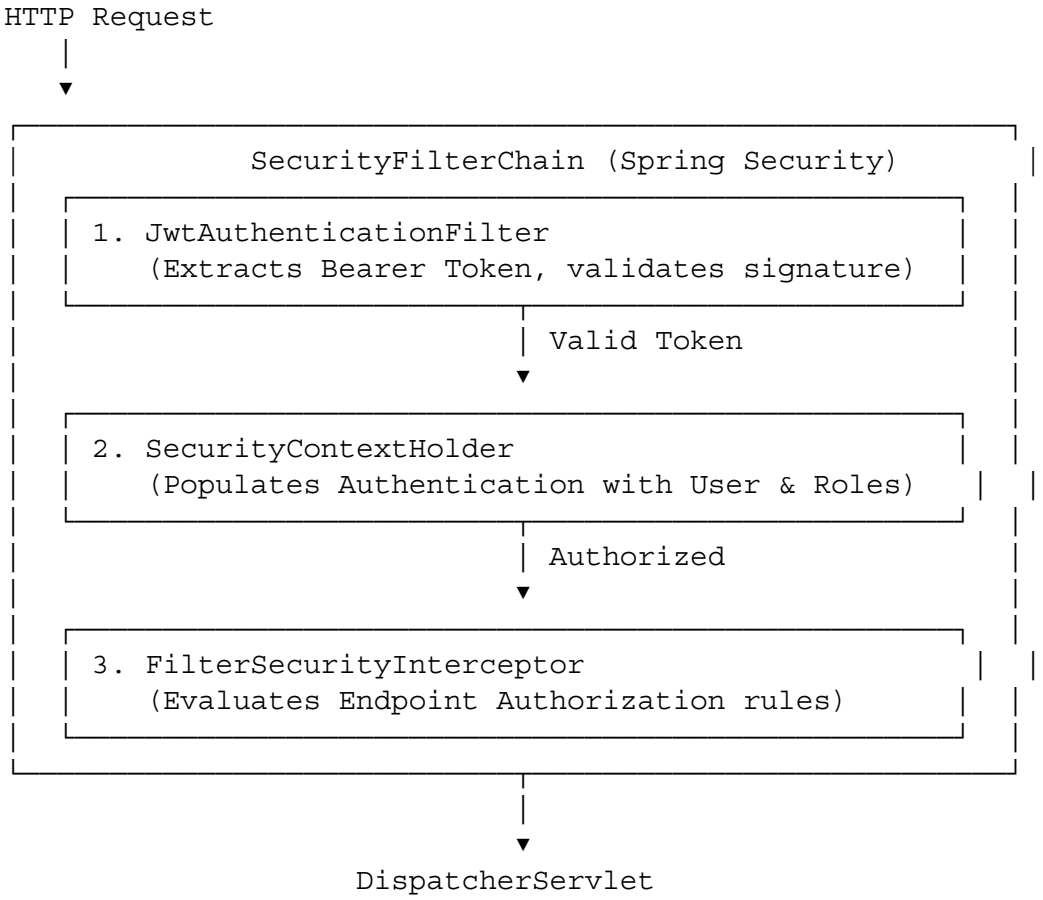
```
.setFieldAccessLevel(org.modelmapper.config.Configuration.AccessLevel.PRIVATE);
        return mapper;
    }
}
```

EmployeeMapperService.java:

```
package com.edtech.ems.service;

import com.edtech.ems.dto.EmployeeDTO;
import com.edtech.ems.model.Employee;
import org.modelmapper.ModelMapper;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class EmployeeMapperService {

    private final ModelMapper modelMapper;

    @Autowired
    public EmployeeMapperService(ModelMapper modelMapper) {
        this.modelMapper = modelMapper;
    }

    public EmployeeDTO convertToDto(Employee employee) {
        return modelMapper.map(employee, EmployeeDTO.class);
    }

    public Employee convertToEntity(EmployeeDTO dto) {
        return modelMapper.map(dto, Employee.class);
    }
}
```

40, 41, 42, 43. Security Architecture (Basic, HTTPS, & JWT)

Architecture & Request Security Flow



40 & 41. Security Basics & HTTPS Configuration

- **HTTPS Setup**: HTTPS encrypts payloads in transit using SSL/TLS.
- **application.properties (Enabling HTTPS with self-signed or PKCS12 Keystore)**:

```
# Enable HTTPS
server.port=8443
server.ssl.key-store=classpath:keystore.p12
server.ssl.key-store-password=changeit
server.ssl.key-store-type=PKCS12
server.ssl.key-alias=ems-alias
```

42 & 43. Complete JWT Authentication Implementation

Maven Dependencies Required:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

```
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.11.5</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
</dependency>
```

JWT Utility Class (JwtTokenProvider.java):

```
package com.edtech.ems.security;

import io.jsonwebtoken.*;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.Authentication;
import org.springframework.stereotype.Component;

import java.security.Key;
import java.util.Date;

@Component
public class JwtTokenProvider {


@Value("${app.jwt-secret:9a2f8c4e7b1a5d3f8e0c2b4a6f8d1e3c5b7a9f0e2d4c6b8a0f2e4d
6c8b0a1f2e}")
    private String jwtSecret;

    @Value("${app.jwt-expiration-milliseconds:86400000}") // 24 hours
    private long jwtExpirationDate;

    private Key key() {
        return Keys.hmacShaKeyFor(jwtSecret.getBytes());
    }

    public String generateToken(Authentication authentication) {
        String username = authentication.getName();
        Date currentDate = new Date();
        Date expireDate = new Date(currentDate.getTime() + jwtExpirationDate);
```

```
        return Jwts.builder()
                .setSubject(username)
                .setIssuedAt(new Date())
                .setExpiration(expireDate)
                .signnWith(key(), SignatureAlgorithm.HS256)
                .compact();
    }

    public String getUsernameFromJWT(String token) {
        Claims claims = Jwts.parserBuilder()
                .setSigningKey(key())
                .build()
                .parseClaimsJws(token)
                .getBody();
        return claims.getSubject();
    }

    public boolean validateToken(String token) {
        try {
            Jwts.parserBuilder().setSigningKey(key()).build().parse(token);
            return true;
        } catch (JwtException | IllegalArgumentException ex) {
            return false;
        }
    }
}
```

JWT Request Authentication Filter (JwtAuthenticationFilter.java):

```
package com.edtech.ems.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.beans.factory.annotation.Autowired;
import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken
;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import
org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.util.StringUtils;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

@Component
```

```
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtTokenProvider tokenProvider;
    private final UserDetailsService userDetailsService;

    @Autowired
    public JwtAuthenticationFilter(JwtTokenProvider tokenProvider,
UserDetailsService userDetailsService) {
        this.tokenProvider = tokenProvider;
        this.userDetailsService = userDetailsService;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws
ServletException, IOException {

        String token = getJwtFromRequest(request);

        if (StringUtils.hasText(token) && tokenProvider.validateToken(token)) {
            String username = tokenProvider.getUsernameFromJWT(token);
            UserDetails userDetails =
userDetailsService.loadUserByUsername(username);

            UsernamePasswordAuthenticationToken authenticationToken =
                    new UsernamePasswordAuthenticationToken(userDetails, null,
userDetails.getAuthorities());
            authenticationToken.setDetails(new
WebAuthenticationDetailsSource().buildDetails(request));


SecurityContextHolder.getContext().setAuthentication(authenticationToken);
        }

        filterChain.doFilter(request, response);
    }

    private String getJwtFromRequest(HttpServletRequest request) {
        String bearerToken = request.getHeader("Authorization");
        if (StringUtils.hasText(bearerToken) && bearerToken.startsWith("Bearer
")) {
            return bearerToken.substring(7);
        }
        return null;
    }
}
```

Spring Security Central Configuration (SecurityConfig.java):

```
package com.edtech.ems.config;

import com.edtech.ems.security.JwtAuthenticationFilter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.config.annotation.authentication.configuration.Aut
henticationConfiguration;
import
org.springframework.security.config.annotation.method.configuration.EnableMetho
dSecurity;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationF
ilter;

@Configuration
@EnableMethodSecurity
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthenticationFilter;

    public SecurityConfig(JwtAuthenticationFilter jwtAuthenticationFilter) {
        this.jwtAuthenticationFilter = jwtAuthenticationFilter;
    }

    @Bean
    public static PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationManager
authenticationManager(AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http.csrf(csrf -> csrf.disable())
            .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
```

```
                .requestMatchers("/api/v1/auth/**", "/swagger-ui/**",
"/v3/api-docs/**").permitAll()
                .requestMatchers("/api/v1/employees/**").hasAnyRole("USER",
"ADMIN")
                .anyRequest().authenticated()
            );

        http.addFilterBefore(jwtAuthenticationFilter,
UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

44. OpenAPI / Swagger Documentation

Definition & Purpose

OpenAPI is a standard specification for describing REST APIs. **Springdoc-openapi** automatically generates interactive API documentation and testing UI directly from Spring Boot annotations.

Maven Dependency:

```
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.3.0</version>
</dependency>
```

Complete OpenAPI Config (OpenApiConfig.java):

```
package com.edtech.ems.config;

import io.swagger.v3.oas.annotations.OpenAPIDefinition;
import io.swagger.v3.oas.annotations.enums.SecuritySchemeType;
import io.swagger.v3.oas.annotations.info.Info;
import io.swagger.v3.oas.annotations.security.SecurityRequirement;
import io.swagger.v3.oas.annotations.security.SecurityScheme;
import org.springframework.context.annotation.Configuration;

@Configuration
@OpenAPIDefinition(
    info = @Info(title = "Employee Management API", version = "v1.0",
description = "Enterprise EMS REST API Documentation"),
    security = @SecurityRequirement(name = "Bearer Authentication")
)
@SecurityScheme(
    name = "Bearer Authentication",
    type = SecuritySchemeType.HTTP,
```

```
    bearerFormat = "JWT",
    scheme = "bearer"
)
public class OpenApiConfig {
}
```

- **Swagger UI Endpoint**: http://localhost:8080/swagger-ui/index.html

45. File Upload & Download API

Purpose

Handling binary payloads (like profile pictures, PDF contracts) associated with employees.

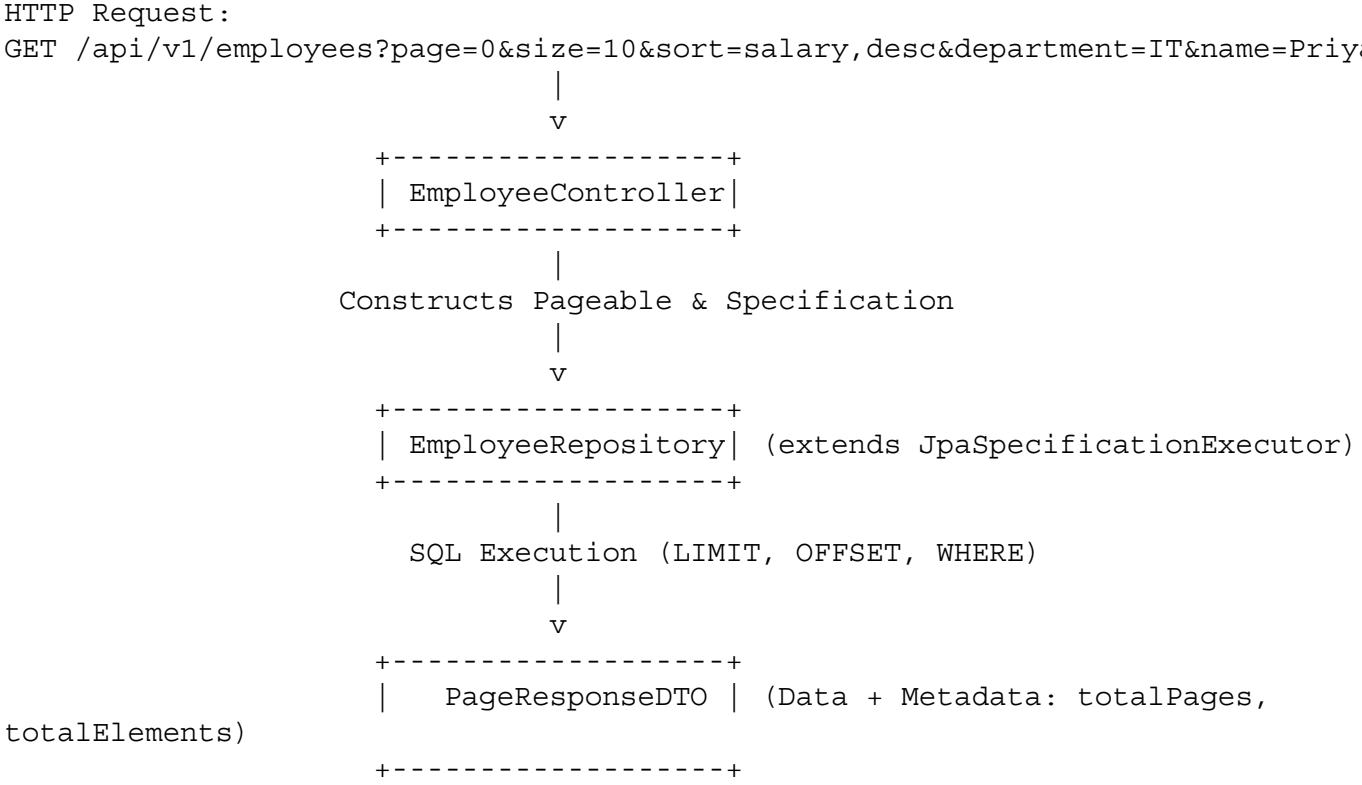
Complete Controller Endpoint (FileAttachmentController.java):

```
package com.edtech.ems.controller;

import org.springframework.core.io.Resource;
import org.springframework.core.io.UrlResource;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;

import java.io.IOException;
import java.nio.file.*;

@RestController
@RequestMapping("/api/v1/employees/files")
public class FileAttachmentController {

    private final Path fileStorageLocation =
Paths.get("uploads").toAbsolutePath().normalize();

    public FileAttachmentController() throws IOException {
        Files.createDirectories(this.fileStorageLocation);
    }

    @PostMapping("/upload")
    public ResponseEntity<String> uploadFile(@RequestParam("file")
MultipartFile file) throws IOException {
        String fileName = System.currentTimeMillis() + "_" +
file.getOriginalFilename();
        Path targetLocation = this.fileStorageLocation.resolve(fileName);
        Files.copy(file.getInputStream(), targetLocation,
StandardCopyOption.REPLACE_EXISTING);
        return ResponseEntity.ok("File uploaded successfully: " + fileName);
    }
```

```
    @GetMapping("/download/{fileName:.+}")
    public ResponseEntity<Resource> downloadFile(@PathVariable String fileName)
throws Exception {
        Path filePath = this.fileStorageLocation.resolve(fileName).normalize();
        Resource resource = new UrlResource(filePath.toUri());

        if(!resource.exists()) {
            throw new RuntimeException("File not found: " + fileName);
        }

        return ResponseEntity.ok()
                .contentType(MediaType.APPLICATION_OCTET_STREAM)
                .header(HttpHeaders.CONTENT_DISPOSITION, "attachment;
filename=\"" + resource.getFilename() + "\"")
                .body(resource);
    }
}
```

46, 47, 48, 49. Advanced Data Operations (Pagination, Sorting, Searching, & Filtering)

Instead of returning unbounded database queries that crash server memory, enterprise APIs implement **Pagination**, **Sorting**, and **Specification-driven Filtering**.



Response Wrapper (PagedResponse.java):

```
package com.edtech.ems.dto;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.util.List;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class PagedResponse<T> {
    private List<T> content;
    private int pageNumber;
    private int pageSize;
    private long totalElements;
    private int totalPages;
    private boolean last;
}
```

Dynamic Specification Search Filter (EmployeeSpecification.java):

```
package com.edtech.ems.repository;

import com.edtech.ems.model.Employee;
import org.springframework.data.jpa.domain.Specification;

public class EmployeeSpecification {

    public static Specification<Employee> hasDepartment(String department) {
        return (root, query, cb) ->
            (department == null || department.isEmpty()) ? cb.conjunction() :
cb.equal(root.get("department"), department);
    }

    public static Specification<Employee> nameContains(String name) {
        return (root, query, cb) ->
            (name == null || name.isEmpty()) ? cb.conjunction() :
cb.like(cb.lower(root.get("firstName")), "%" + name.toLowerCase() + "%");
    }
}
```

Repository Integration:

```
package com.edtech.ems.repository;

import com.edtech.ems.model.Employee;
import org.springframework.data.jpa.repository.JpaRepository;
```

```
import org.springframework.data.jpa.repository.JpaSpecificationExecutor;
import org.springframework.stereotype.Repository;

@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long>,
JpaSpecificationExecutor<Employee> {
}
```

Service Search Implementation (EmployeeSearchService.java):

```
package com.edtech.ems.service;

import com.edtech.ems.dto.EmployeeDTO;
import com.edtech.ems.dto.PagedResponse;
import com.edtech.ems.model.Employee;
import com.edtech.ems.repository.EmployeeRepository;
import com.edtech.ems.repository.EmployeeSpecification;
import org.modelmapper.ModelMapper;
import org.springframework.data.domain.*;
import org.springframework.data.jpa.domain.Specification;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.stream.Collectors;

@Service
public class EmployeeSearchService {

    private final EmployeeRepository employeeRepository;
    private final ModelMapper modelMapper;

    public EmployeeSearchService(EmployeeRepository employeeRepository,
ModelMapper modelMapper) {
        this.employeeRepository = employeeRepository;
        this.modelMapper = modelMapper;
    }

    public PagedResponse<EmployeeDTO> searchEmployees(int page, int size,
String sortBy, String sortDir, String department, String name) {
        Sort sort = sortDir.equalsIgnoreCase(Sort.Direction.ASC.name()) ?
Sort.by(sortBy).ascending() : Sort.by(sortBy).descending();
        Pageable pageable = PageRequest.of(page, size, sort);

        Specification<Employee> spec =
Specification.where(EmployeeSpecification.hasDepartment(department))

.and(EmployeeSpecification.nameContains(name));

        Page<Employee> employees = employeeRepository.findAll(spec, pageable);

        List<EmployeeDTO> content = employees.getContent().stream()
```

```
                .map(emp -> modelMapper.map(emp, EmployeeDTO.class))
                .collect(Collectors.toList());

        return new PagedResponse<>(
                content,
                employees.getNumber(),
                employees.getSize(),
                employees.getTotalElements(),
                employees.getTotalPages(),
                employees.isLast()
        );
    }
}
```

50. Spring Boot Best Practices Summary Matrix

Domain	Industry Standard Practice	Why it Matters
Architecture	Enforce clear boundary layer separation (DTOs ↔ Service ↔ Entities).	Prevents database structures from leaking to external consumers.
Configuration	Store credentials in environment variables/vaults, not in application.properties.	Prevents source code credential leaks in Git repositories.
Security	Stateless JWT authentication + Role-Based Access Control (RBAC).	Highly scalable horizontal deployment across clusters without sticky sessions.
Performance	Always enforce default limit/size parameters on search endpoints.	Prevents OutOfMemoryError caused by loading millions of records at once.
Database	Avoid spring.jpa.hibernate.ddl-auto =update in production. Use Flyway or Liquibase.	Guarantees controlled, audited, and rollback-safe database migrations.

Complete Production Architecture & Blueprint

Complete Folder Structure

```
ems-backend/
├── pom.xml
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/edtech/ems/
│   │   │       ├── EmsApplication.java
│   │   │       ├── config/
│   │   │       │   ├── ModelMapperConfig.java
│   │   │       │   ├── OpenApiConfig.java
│   │   │       │   └── SecurityConfig.java
│   │   │       ├── controller/
│   │   │       │   ├── AuthController.java
│   │   │       │   ├── EmployeeController.java
│   │   │       │   └── FileAttachmentController.java
│   │   │       ├── dto/
│   │   │       │   ├── EmployeeDTO.java
│   │   │       │   ├── ErrorDetails.java
│   │   │       │   ├── JwtAuthResponse.java
│   │   │       │   ├── LoginDTO.java
│   │   │       │   └── PagedResponse.java
│   │   │       ├── exception/
│   │   │       │   ├── GlobalExceptionHandler.java
│   │   │       │   └── ResourceNotFoundException.java
│   │   │       ├── model/
│   │   │       │   ├── Employee.java
│   │   │       │   └── User.java
│   │   │       ├── repository/
│   │   │       │   ├── EmployeeRepository.java
│   │   │       │   └── EmployeeSpecification.java
│   │   │       ├── security/
│   │   │       │   ├── CustomUserDetailsService.java
│   │   │       │   ├── JwtAuthenticationFilter.java
│   │   │       │   └── JwtTokenProvider.java
│   │   │       └── service/
│   │   │           ├── EmployeeService.java
│   │   │           └── impl/
│   │   │               └── EmployeeServiceImpl.java
│   │   └── resources/
│   │       ├── application.properties
│   │       ├── application-prod.properties
│   │       ├── db/migration/V1__init_schema.sql
│   │       └── logback-spring.xml
│   └── test/
│       └── java/com/edtech/ems/
│           ├── controller/EmployeeControllerTest.java
│           └── service/EmployeeServiceTest.java
```

Enterprise Deployment Checklist

1. **Environment-Specific Profiles**: Separate application-dev.properties, application-prod.properties.
2. **Database Migrations**: Integrated **Flyway** / **Liquibase** scripts.
3. **Health Check & Metrics**: Include spring-boot-starter-actuator with endpoints exposed securely (/actuator/health, /actuator/metrics).
4. **Containerization**: Use multi-stage Dockerfiles:

```
# Build Stage
FROM maven:3.9.5-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

# Run Stage
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```